Assignment 2 — Simple Search Engine

Nikita Shiyanov, AI-02

1 Methodology

I built a search engine that indexes Wikipedia articles and retrieves relevant documents for a query using BM25. The system runs in Docker with two nodes: `master` and `slave1`. The master runs HDFS NameNode, YARN ResourceManager, Cassandra. Slave1 runs DataNode and NodeManager. I used Hadoop 3.3.6, Spark 3.5.0, Cassandra 4.1.3.

1.1 Data Preparation

The script `prepare_data.py` reads a parquet file with PyArrow in batches (to avoid running out of memory), and collects 100 documents. For each document it creates a `.txt` file named `<doc_id>_<doc_title>.txt` and uploads it to HDFS at `/data/`. Then it uses PySpark RDD to read all files from HDFS, extract id, title and text, and save as one partition to `/input/data` in tab-separated format.

1.2 Indexer

Three MapReduce pipelines:

- **Pipeline 1:** `mapper1.py` tokenizes each document, removes stopwords, counts word occurrences. `reducer1.py` groups by word and counts the number of documents this word appears in (df). Output: `/tmp/indexer/pipeline1`.
- **Pipeline 2:** `mapper2.py` computes length of each document. `reducer2.py` is a single reducer to get global stats — total docs N and average doc length $\overline{dl}$. Output: `/indexer/doc_stats`.
- **Pipeline 3:** `mapper3.py` passes data through, Hadoop sorts by word. `reducer3.py` groups all postings for each word into compact format `word<TAB>df<TAB>doc_id:tf:dl,...`. Output: `/indexer/inverted_index`.

1.3 Cassandra Tables

Three tables in keyspace `search_engine`:

- `inverted_index`: Primary key (term, doc_id). Stores term, doc_id, tf, df, doc_length. The partition key is term so we can quickly get all docs for a word.
- `doc_stats`: Primary key doc_id. Stores doc_id, doc_title, doc_length.
- `corpus_stats`: id=1. Stores num_docs, avg_dl, total_dl.

Script `store_index_to_cassandra.py` reads index data from HDFS with PySpark and inserts into these tables.

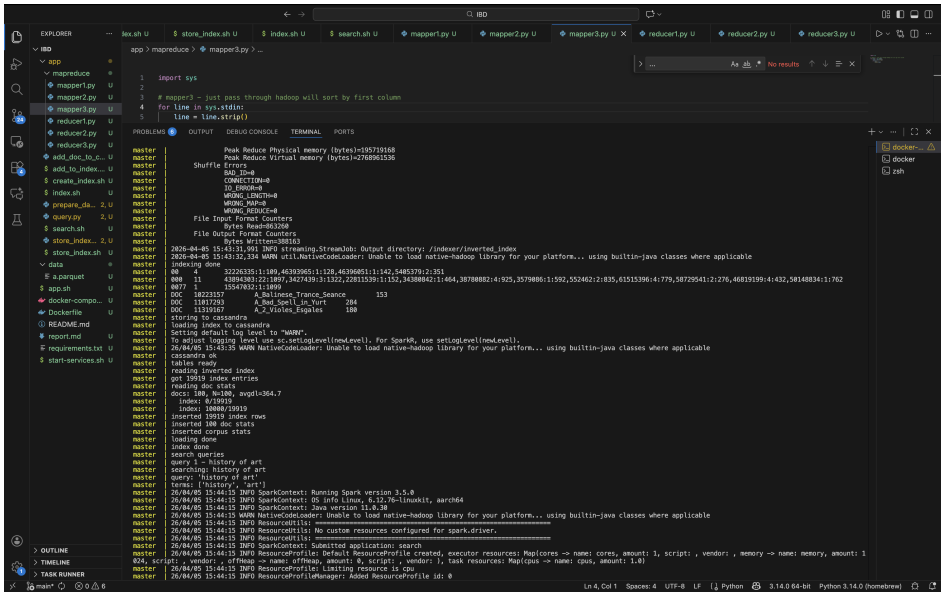

Figure 1: Successful indexing of 100 docs

1.4 Ranker

Script `query.py` tokenizes the user query in the same way as the indexer, gets corpus stats and postings from Cassandra, then uses PySpark RDD to compute BM25 score for each document.

$$BM25(q,d) = \sum_{\text{term } t \in q} \log\left(\frac{N}{df(t)}\right) \cdot \frac{(k_1+1)tf(t,d)}{k_1\left((1-b)+b\cdot\frac{dl(d)}{\overline{dl}}\right)+tf(t,d)}$$

where $k_1 = 1.5$, $b = 0.75$. Returns the top 10 documents sorted by score.

1.5 Adding New Documents

Script `add_to_index.sh` takes a file, uploads it to HDFS, runs MapReduce on it, and updates Cassandra tables — adds new doc_stats, updates corpus_stats with new N and $\overline{dl}$, inserts new postings and updates `df` for affected terms.

2 Demonstration

2.1 How to Run

1. Put `a.parquet` file in the `./data/` folder.
2. Run `docker compose up`
3. System starts services, prepares 100 documents, indexes them, runs test queries.

2.2 Indexing

The system indexed 100 documents, created 19919 index entries. Corpus statistics: $N = 100$, average document length = 364.7 tokens.

#	DocID	Score	Title
1	28380942	4.7527	A P Food Stores Building
2	9279736	4.2279	A Cass McCombs album
3	165709	3.8051	A Jethro Tull album
4	38370970	3.4222	A Jimmy Raney album
5	38780882	3.0645	A Agnetha Faltskog album
6	3579086	3.0597	A C Black
7	58729541	2.9859	A Usher and Zaytoven album
8	748872	2.8078	A Cyrillic
9	51755157	2.3390	A Bachelors Baby
10	62397582	2.3304	A F Harvey Brothers

2.3 Query Results

Query: `history of art`

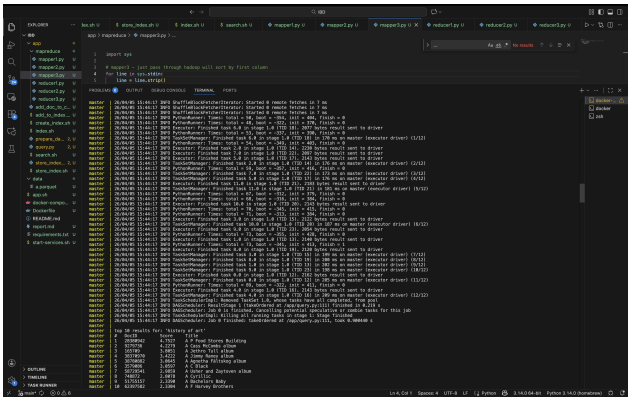

Figure 2: Results for "history of art"

Top results mention art and history topics in their text content. With only 100 docs, results are limited but the ranking works correctly.

Query: `computer science algorithms`

#	DocID	Score	Title
1	48227028	15.7557	A B M Shawkat Ali
2	11506718	5.8385	A B Wood Medal
3	3579086	4.1731	A C Black

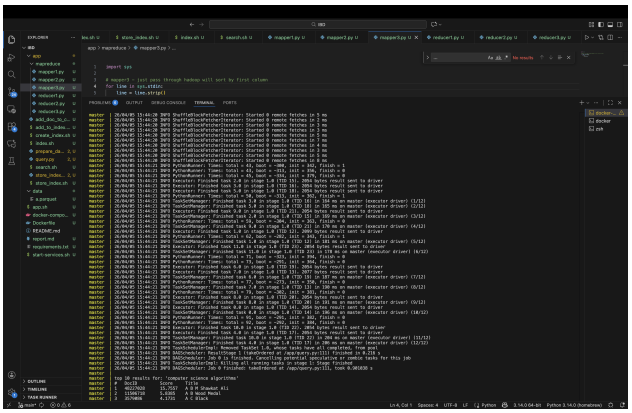

Figure 3: Results for "computer science algorithms"

The first result has much higher score because the article mentions all three query terms. Multi-term query adds up BM25 scores from each word.

Query: `ancient civilization`

#	DocID	Score	Title
1	63462998	3.8254	A Ballad About Green Wood
2	3579086	3.0550	A C Black

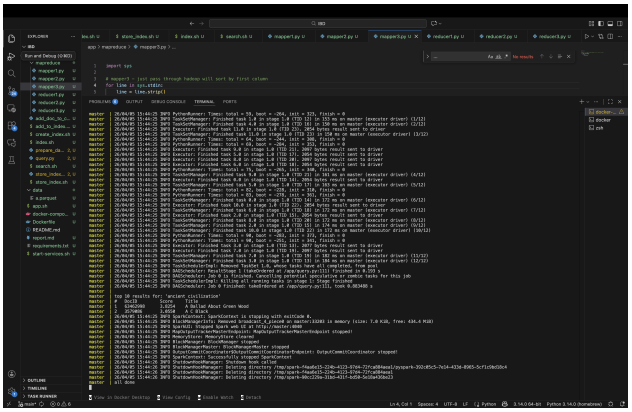

Figure 4: Results for "ancient civilization"

Few results because with only 100 docs, not many contain these terms.

2.4 Findings

BM25 handles document length well — long documents do not get higher scores just because they have more words. Stopword removal reduces noise and index size. The saturation from k_1 prevents documents from scoring too high by repeating terms. With a small corpus of 100 docs, some queries return few results, which is expected.